

CS4225/CS5425 Big Data Systems for Data Science

Spark I: Basics

Ai Xin
School of Computing
National University of Singapore
aixin@comp.nus.edu.sg



Intro

- Lecturer: Ai Xin
 - Email: aixin@nus.edu.sg
 - Lecture Questions: Post your questions to Canvas/Discussion or Email me
 - Office Hours: Friday 4pm-5pm (details will be released in announcement)
- TAs
 - Assignment 2 (Post your questions to Canvas/Discussion or Email TAs)

TAs	Email	Canvas Name Initials
RYAN RENO LIM	ryan.reno@u.nus.edu	A-E
SHEN LEYANG	lshen@u.nus.edu	F-K
WANG RUIYU	e1322751@u.nus.edu	L-N
ZHENG JINGNAN	jingnan.zheng@u.nus.edu	R-T
ZHENG YING	zheng.ying@u.nus.edu	O-Q, V-Z

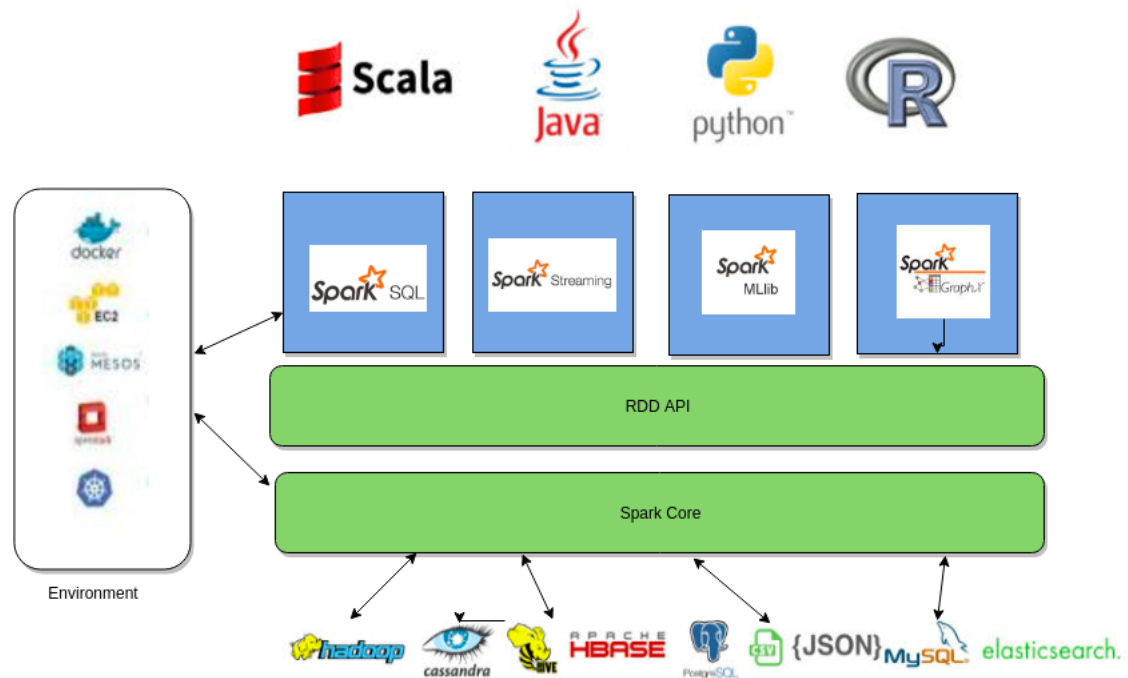
- Tutorials (Post your questions to Canvas/Discussion or Email TA)
 - DESMOND LAU CHEE LOONG dlau@nus.edu.sg

Schedule

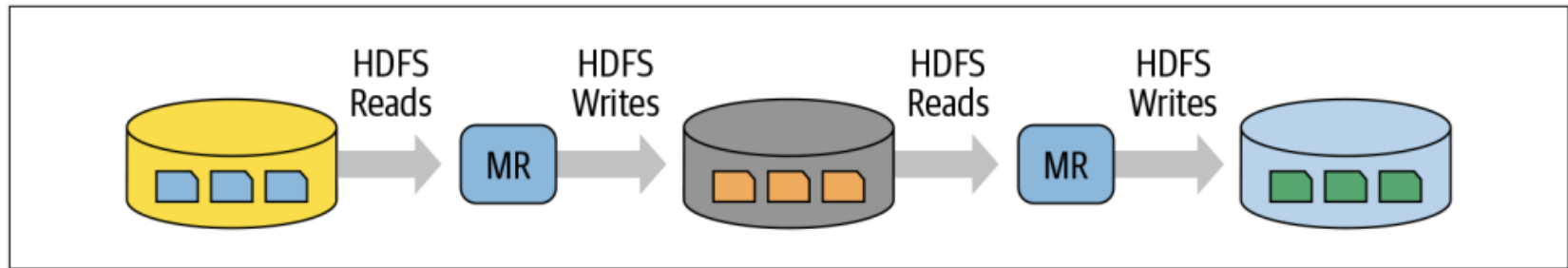
Week	Date	Topics	Tutorial	Due Dates
1	15-Jan	Course Overview and Introduction		
2	22-Jan	Principles of Big Data Systems		
3	29-Jan	MapReduce/Hadoop- Intro		
4	5-Feb	MapReduce- Performance Analysis	Assignment 1 Briefing	Assignment 1 released
5	12-Feb	MapReduce- Database and Data Mining	Tutorial: MapReduce	
6	19-Feb	NoSQL Overview and Revision		
Recess				
7	5 Mar	Apache Spark 1		Assignment 1 due (8 Mar 11.59pm), Assignment 2 released
8	12 Mar	Apache Spark 2	Assignment 2 Briefing	Interview of Assignment 1
9	19 Mar	Stream Processing 1	Tutorial: Spark	
10	26 Mar	Stream Processing 2	Tutorial: Stream Processing	
11	2 Apr	NUS Well-Being Day		
12	9 Apr	Large Graph Processing 1		Assignment 2 due (12 Apr 11.59pm)
13	16 Apr	Large Graph Processing 2	Tutorial: Graph Processing	Interview of Assignment 2
	29 Apr	Final Exam		

Today's Plan

- Introduction and Basics
- Working with RDDs
- Caching and DAGs
- DataFrames and Datasets

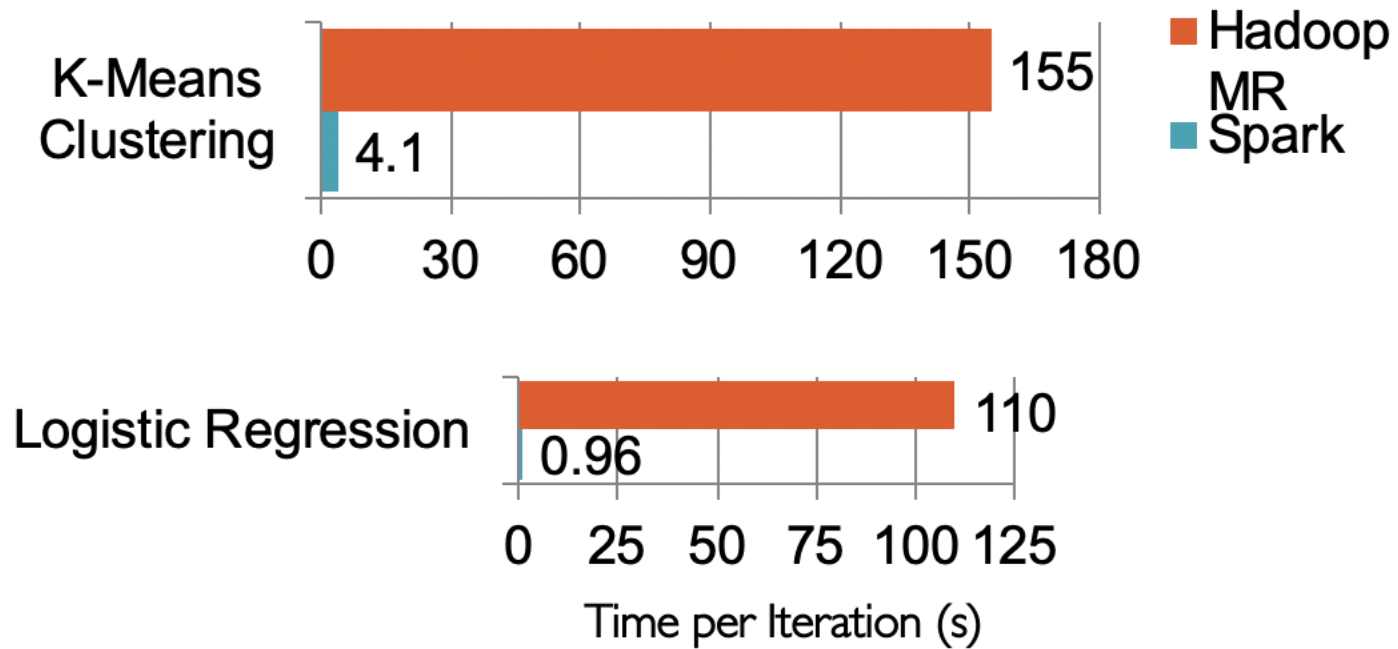


Motivation: Hadoop vs Spark



- Issues with Hadoop Mapreduce:
 - **Network and disk I/O costs:** intermediate data has to be written to local disks and shuffled across machines, which is slow
 - Not suitable for **iterative** (i.e. modifying small amounts of data repeatedly) processing, such as interactive workflows, as each individual step has to be modelled as a MapReduce job.
- Spark stores most of its intermediate results in memory, making it much faster, especially for iterative processing
 - When memory is insufficient, Spark **spills to disk** which requires disk I/O

Performance Comparison



Ease of Programmability

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable>{

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context
            ) throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }
}
```

```
public static class IntSumReducer
    extends Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
        Context context
        ) throws IOException, InterruptedException {

        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```

WordCount (Hadoop MapReduce)

Ease of Programmability

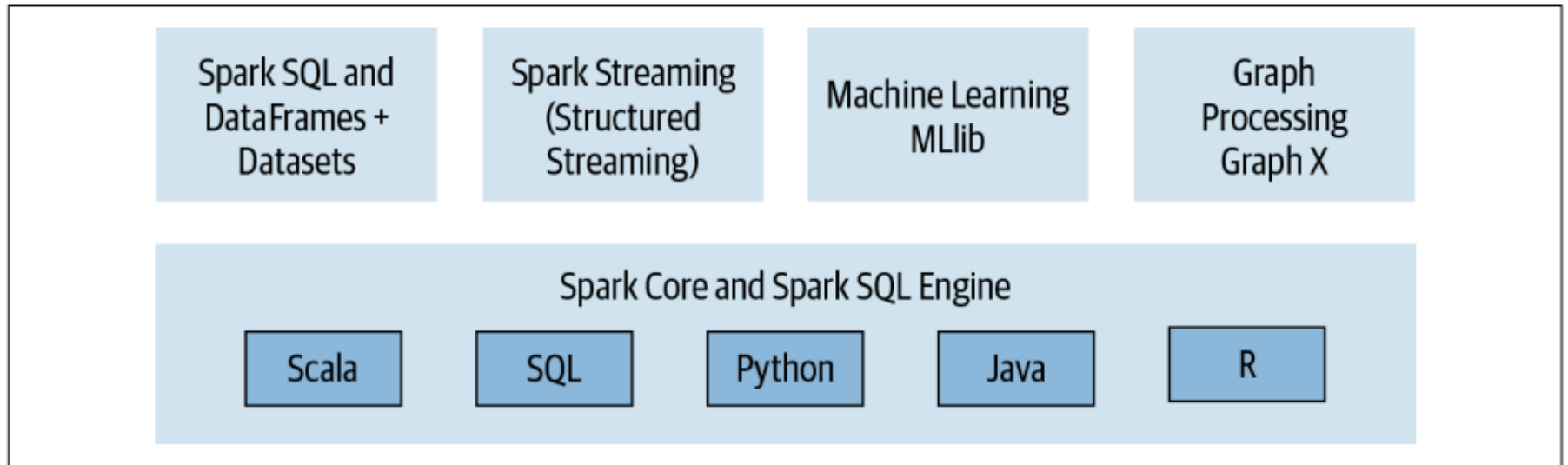
```
val file = sc.textFile("hdfs://...")

val counts = file.flatMap(line => line.split(" "))
                   .map(word => (word, 1))
                   .reduceByKey(_ + _)

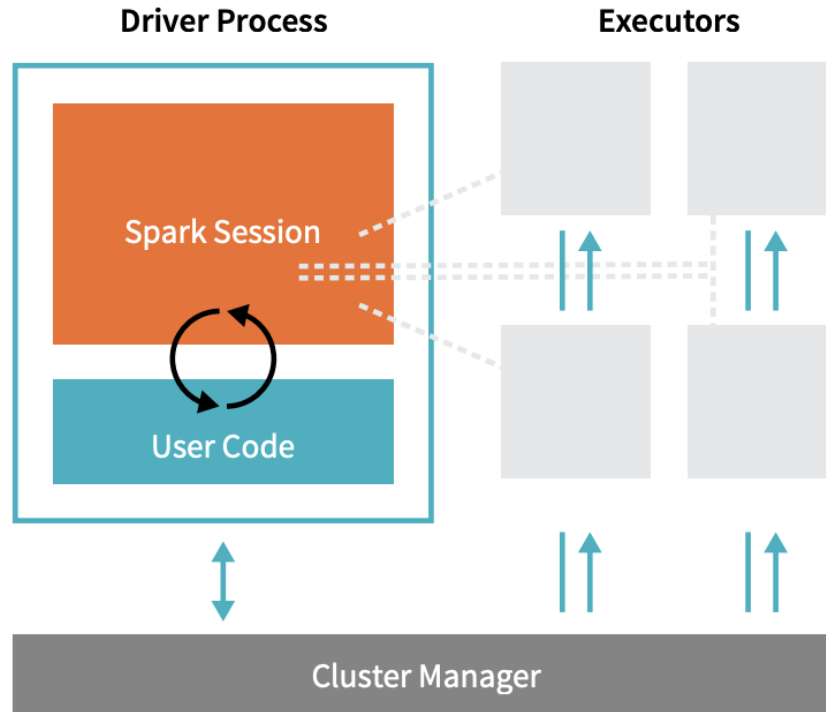
counts.save("...")
```

WordCount (Spark)

Spark Components and API Stack

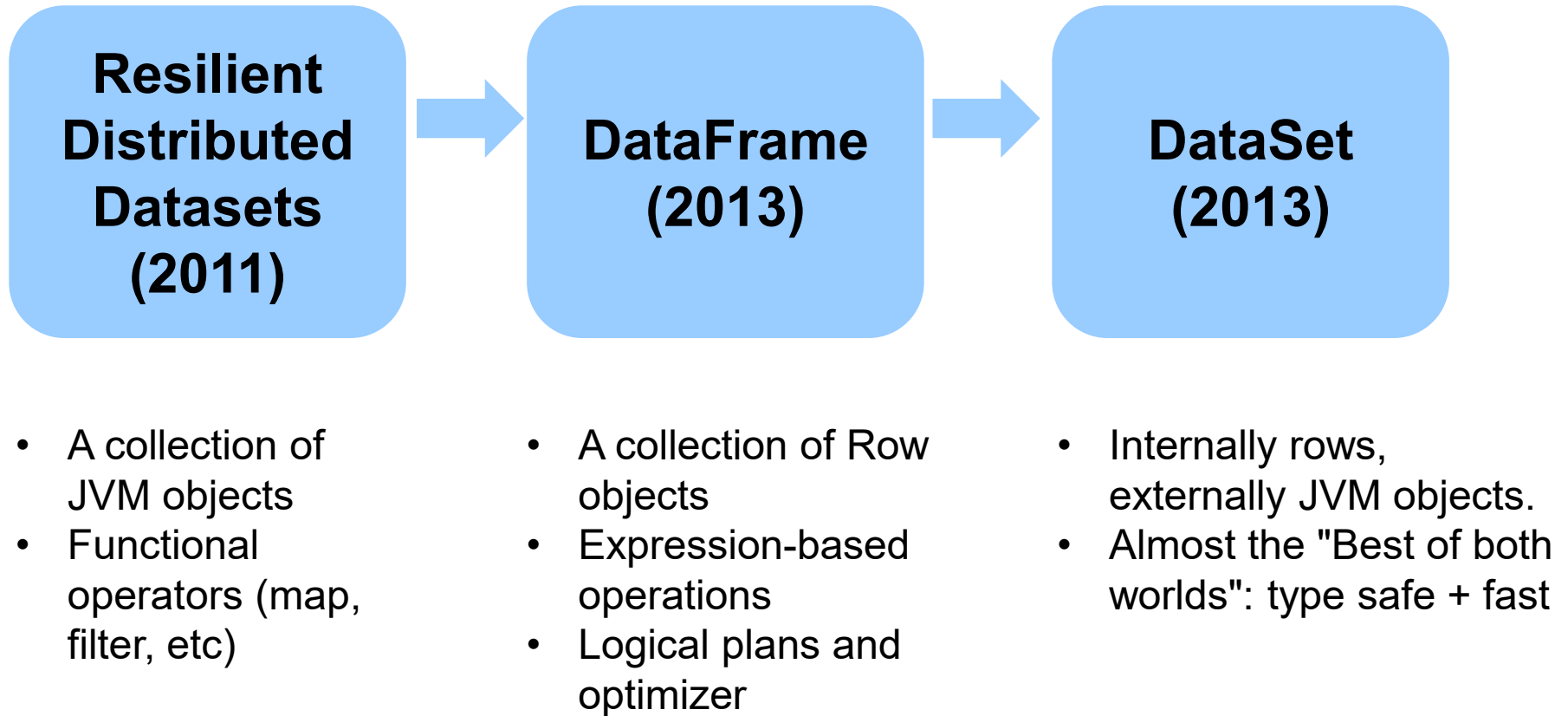


Spark Architecture



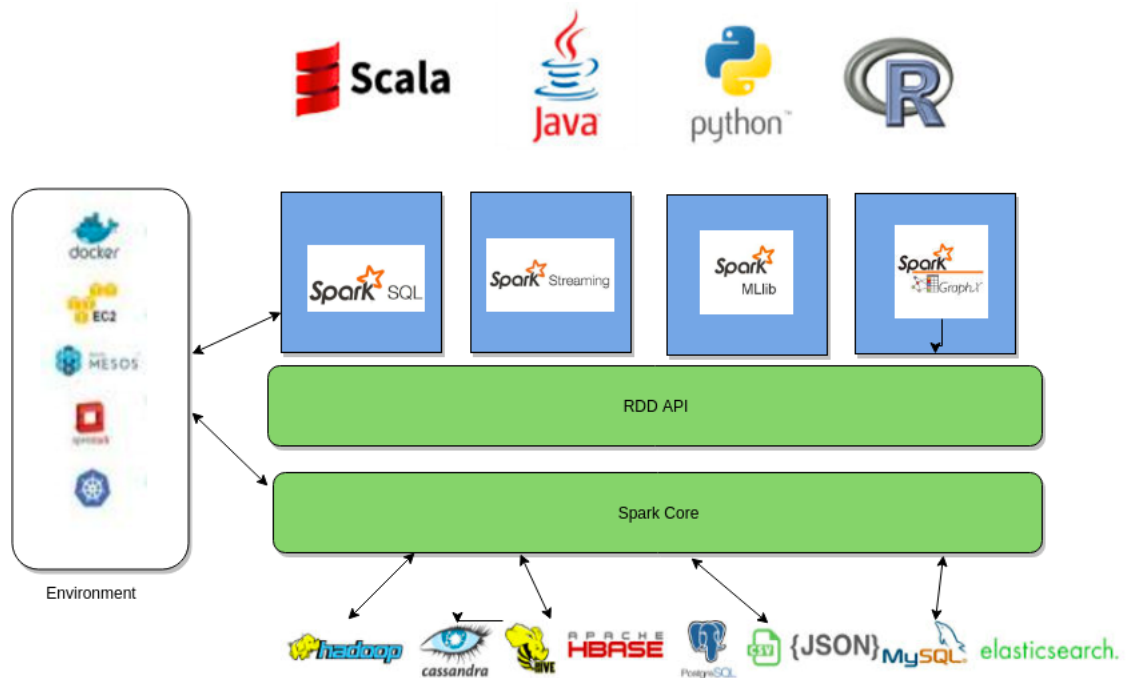
- **Driver Process** responds to user input, manages the Spark application etc., and distributes work to **Executors**, which run the code assigned to them and send the results back to the driver
- **Cluster Manager** (can be Spark's standalone cluster manager, YARN, Mesos or Kubernetes) allocates resources when the application requests it
- In **local mode**, all these processes run on the same machine

Evolution of Spark APIs



Today's Plan

- Introduction and Basics
- **Working with RDDs**
- Caching and DAGs
- DataFrames and Datasets



Achieve fault tolerance
through **lineages**



Represent a collection of
objects that is **distributed over
machines**



Resilient Distributed Datasets (RDDs)

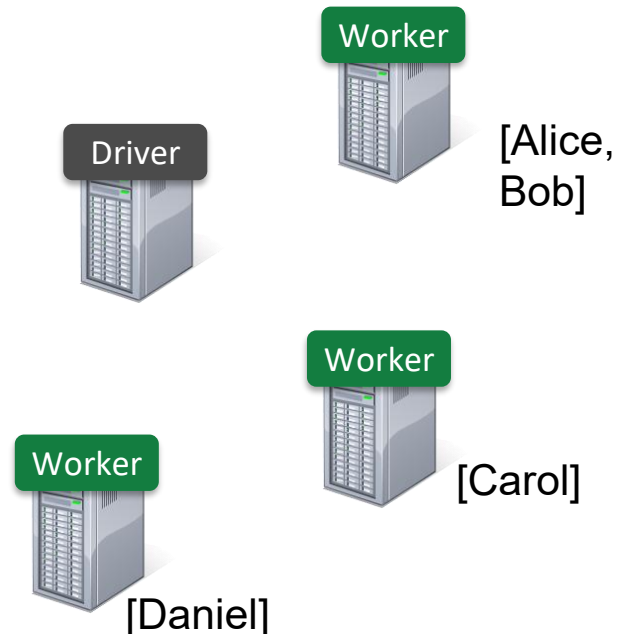
RDD: Distributed Data

```
# Create an RDD of names, distributed over 3 partitions
```

```
dataRDD = sc.parallelize(["Alice", "Bob", "Carol", "Daniel"], 3)
```

Partition data
into 3 parts

- RDDs are **immutable**, i.e. they cannot be changed once created.
- This is an RDD with 4 strings. In actual hardware, it will be partitioned into the 3 workers.





Transformations

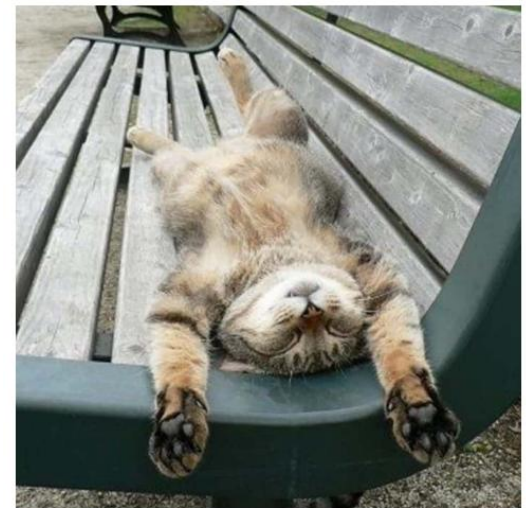
- **Transformations** are a way of transforming RDDs into RDDs.

```
# Create an RDD: length of names
```

```
dataRDD = sc.parallelize(["Alice", "Bob", "Carol", "Daniel"], 3)
```

```
nameLen = dataRDD.map(lambda s: len(s))
```

- This represents the transformation that maps each string to its length, creating a new RDD.
- However, transformations are **lazy**. This means the transformation will not be executed yet, until an **action** is called on it
 - Q: what are the advantages of being lazy?
 - A: Spark can optimize the query plan to improve speed (e.g. removing unneeded operations)
- Examples of transformations: `map`, `order`, `groupBy`, `filter`, `join`, `select`



Actions

- **Actions** trigger Spark to compute a result from a series of transformations.

```
dataRDD = sc.parallelize(["Alice", "Bob", "Carol", "Daniel"], 3)
nameLen = dataRDD.map(lambda s: len(s))
nameLen.collect()
```

```
[5, 3, 5, 6]
```

- `collect()` here is an action.
 - It is the action that asks Spark to retrieve all elements of the RDD to the driver node.
- Examples of actions: `show`, `count`, `save`, `collect`

Distributed Processing

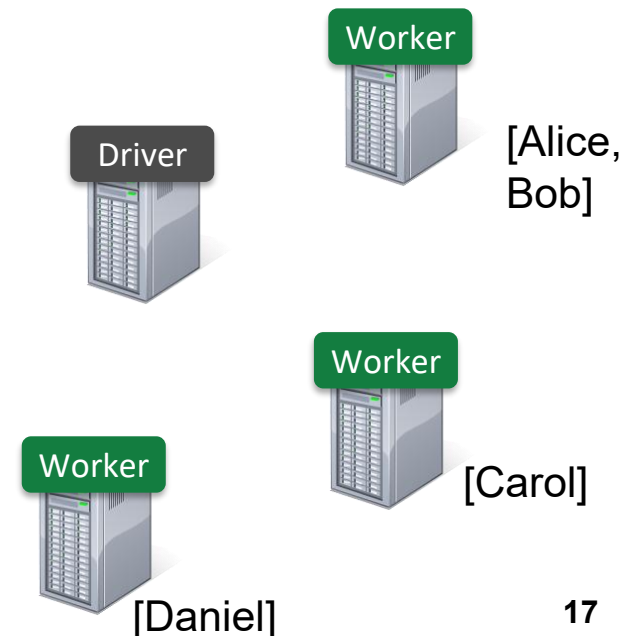
```
# Create an RDD: length of names
```

```
dataRDD = sc.parallelize(["Alice", "Bob", "Carol", "Daniel"], 3)
```

```
nameLen = dataRDD.map(lambda s: len(s))
```

```
nameLen.collect()
```

- As we previously said, RDDs are actually distributed across machines.
- Thus, the transformations and actions are executed in parallel. The results are only sent to the driver in the final step.



Distributed Processing

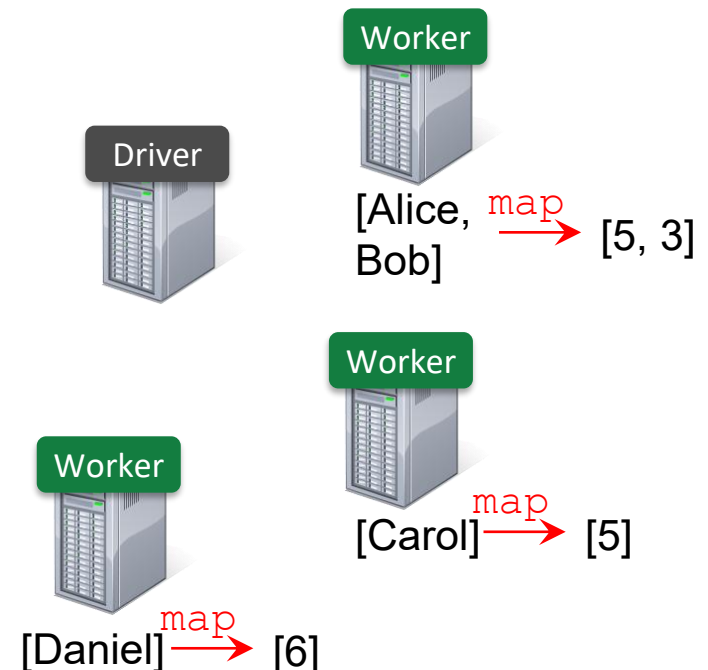
```
# Create an RDD: length of names
```

```
dataRDD = sc.parallelize(["Alice", "Bob", "Carol", "Daniel"], 3)
```

```
nameLen = dataRDD.map(lambda s: len(s))
```

```
nameLen.collect()
```

- As we previously said, RDDs are actually distributed across machines.
- Thus, the transformations and actions are executed in parallel. The results are only sent to the driver in the final step.



Distributed Processing

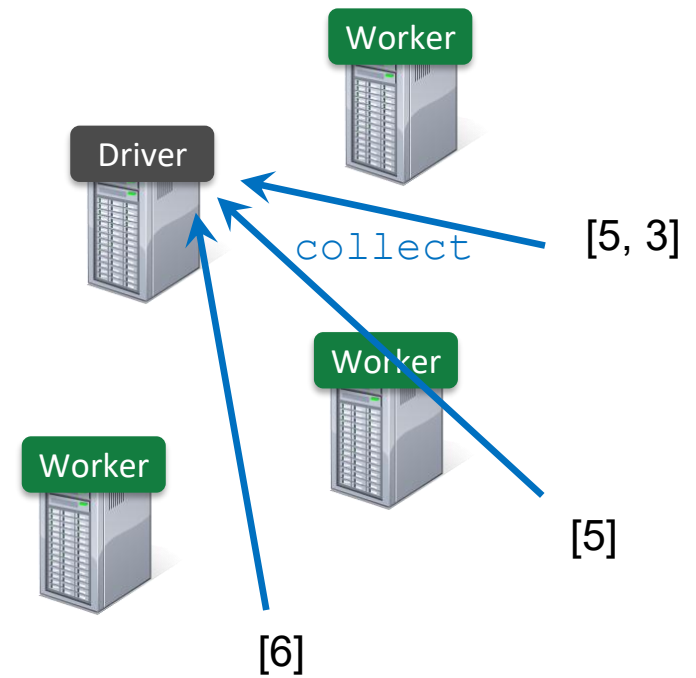
```
# Create an RDD: length of names
```

```
dataRDD = sc.parallelize(["Alice", "Bob", "Carol", "Daniel"], 3)
```

```
nameLen = dataRDD.map(lambda s: len(s))
```

```
nameLen.collect()
```

- As we previously said, RDDs are actually distributed across machines.
- Thus, the transformations and actions are executed in parallel. The results are only sent to the driver in the final step.



Distributed Processing

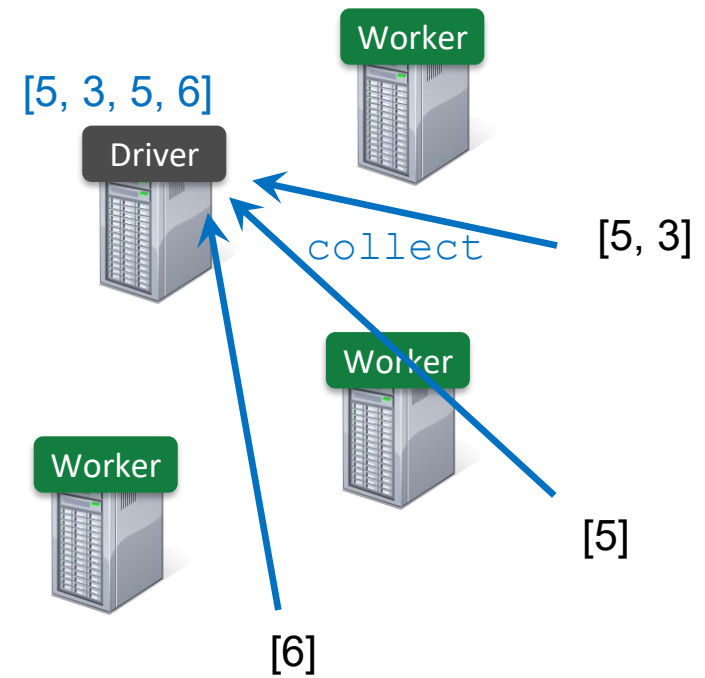
```
# Create an RDD: length of names
```

```
dataRDD = sc.parallelize(["Alice", "Bob", "Carol", "Daniel"], 3)
```

```
nameLen = dataRDD.map(lambda s: len(s))
```

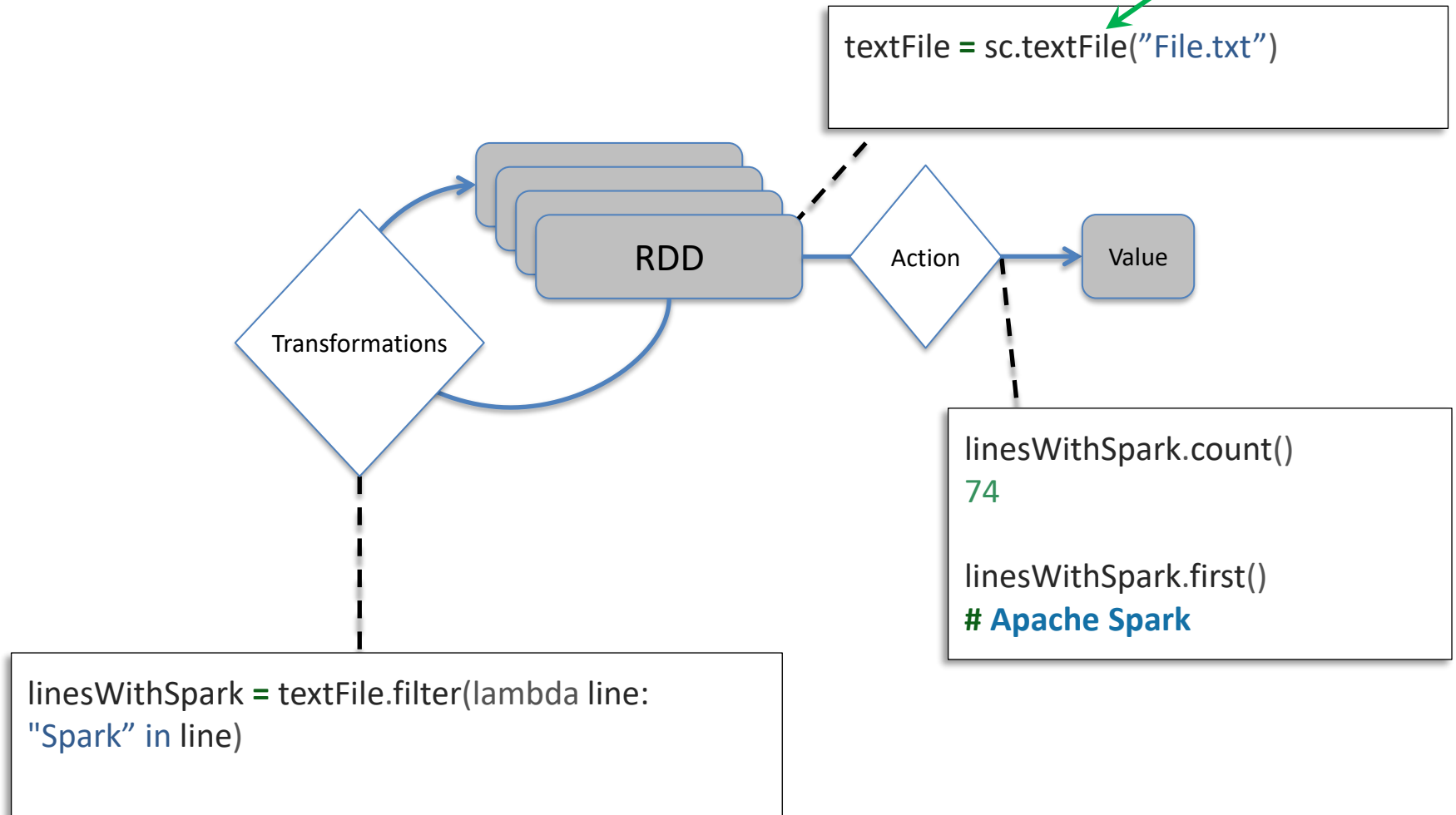
```
nameLen.collect()
```

- As we previously said, RDDs are actually distributed across machines.
- Thus, the transformations and actions are executed in parallel. The results are only sent to the driver in the final step.



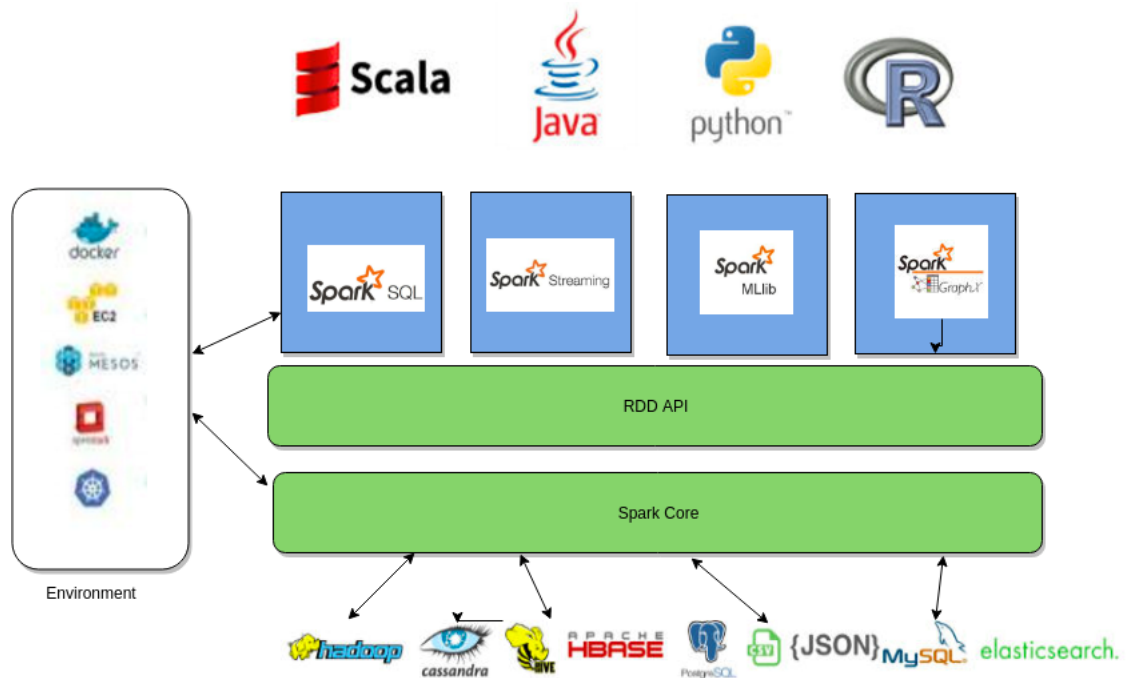
Working with RDDs

Note: this reads the file on each worker node in parallel, not on the driver node



Today's Plan

- Introduction and Basics
- Working with RDDs
- **Caching and DAGs**
- DataFrames and Datasets



Caching

Log Mining example: Load error messages from a log into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda s: s.startswith("ERROR"))
messages = errors.map(lambda s: s.split("\t")[2])
messages.cache()
```

```
messages.filter(lambda s: "mysql" in s).count()
```



Caching

Log Mining example: Load error messages from a log into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")  
errors = lines.filter(lambda s: s.startswith("ERROR"))  
messages = errors.map(lambda s: s.split("\t")[2])  
messages.cache()
```

```
messages.filter(lambda s: "mysql" in s).count()
```

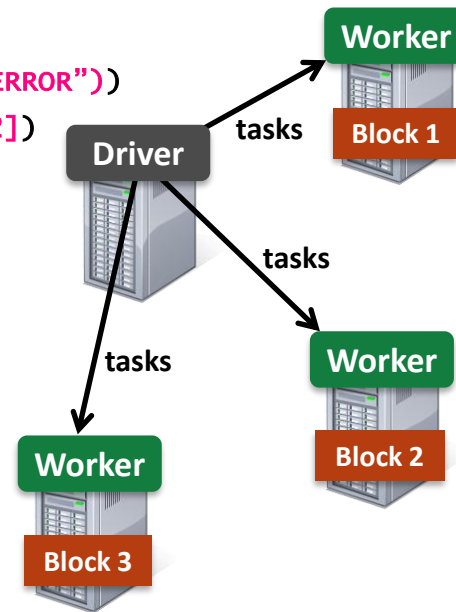


Caching

Log Mining example: Load error messages from a log into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda s: s.startswith("ERROR"))
messages = errors.map(lambda s: s.split("\t")[2])
messages.cache()
```

```
messages.filter(lambda s: "mysql" in s).count()
```

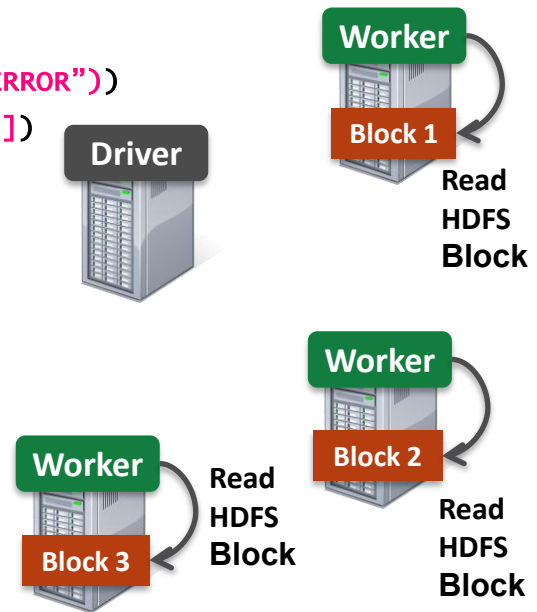


Caching

Log Mining example: Load error messages from a log into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")  
errors = lines.filter(lambda s: s.startswith("ERROR"))  
messages = errors.map(lambda s: s.split("\t")[2])  
messages.cache()
```

```
messages.filter(lambda s: "mysql" in s).count()
```

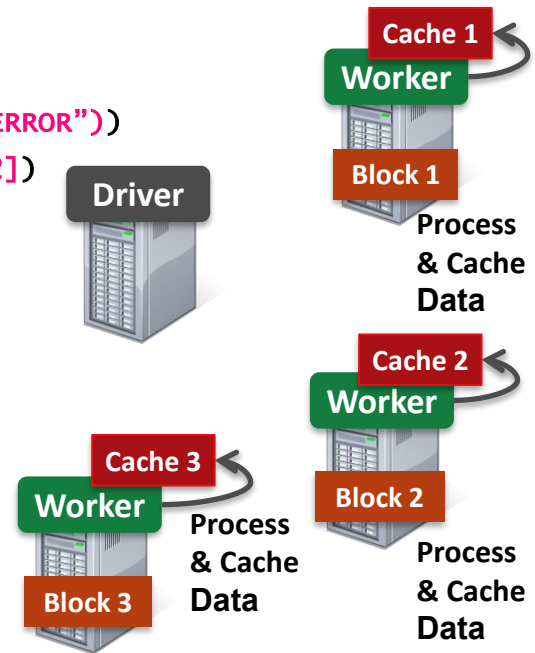


Caching

Log Mining example: Load error messages from a log into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda s: s.startswith("ERROR"))
messages = errors.map(lambda s: s.split("\t")[2])
messages.cache()
```

```
messages.filter(lambda s: "mysql" in s).count()
```

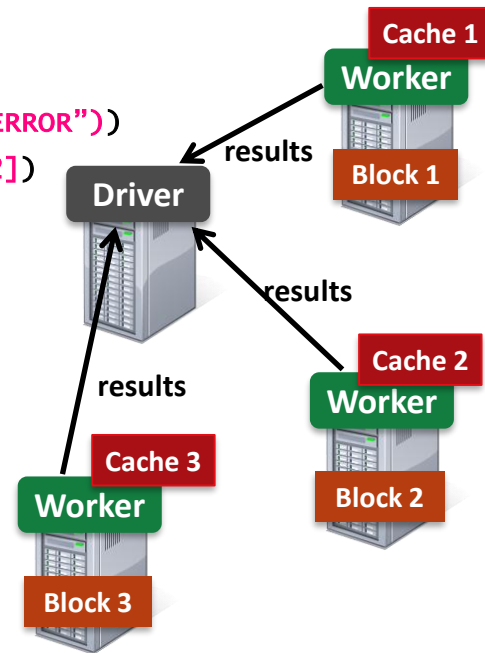


Caching

Log Mining example: Load error messages from a log into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda s: s.startswith("ERROR"))
messages = errors.map(lambda s: s.split("\t")[2])
messages.cache()
```

```
messages.filter(lambda s: "mysql" in s).count()
```

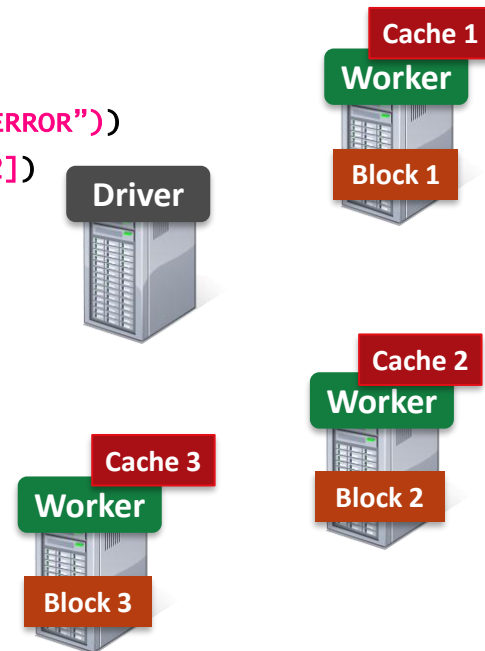


Caching

Log Mining example: Load error messages from a log into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda s: s.startswith("ERROR"))
messages = errors.map(lambda s: s.split("\t")[2])
messages.cache()
```

```
messages.filter(lambda s: "mysql" in s).count()
messages.filter(lambda s: "php" in s).count()
```

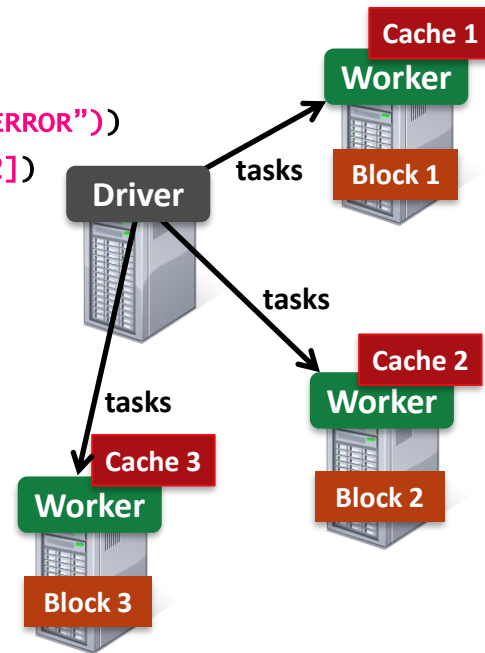


Caching

Log Mining example: Load error messages from a log into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda s: s.startswith("ERROR"))
messages = errors.map(lambda s: s.split("\t")[2])
messages.cache()
```

```
messages.filter(lambda s: "mysql" in s).count()
messages.filter(lambda s: "php" in s).count()
```

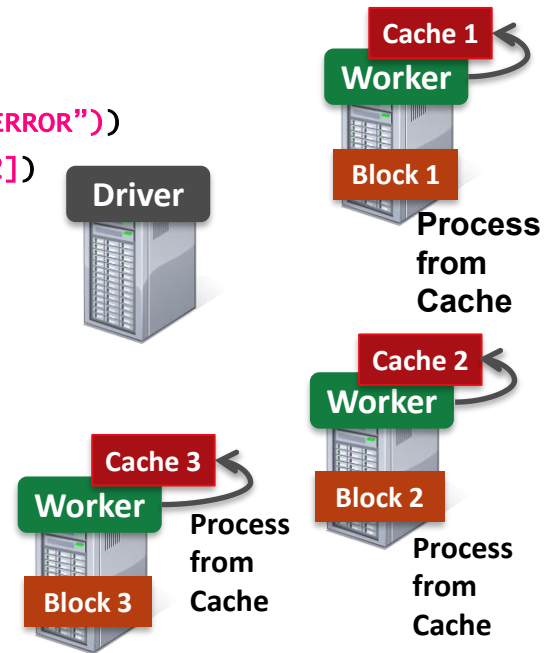


Caching

Log Mining example: Load error messages from a log into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda s: s.startswith("ERROR"))
messages = errors.map(lambda s: s.split("\t")[2])
messages.cache()
```

```
messages.filter(lambda s: "mysql" in s).count()
messages.filter(lambda s: "php" in s).count()
```

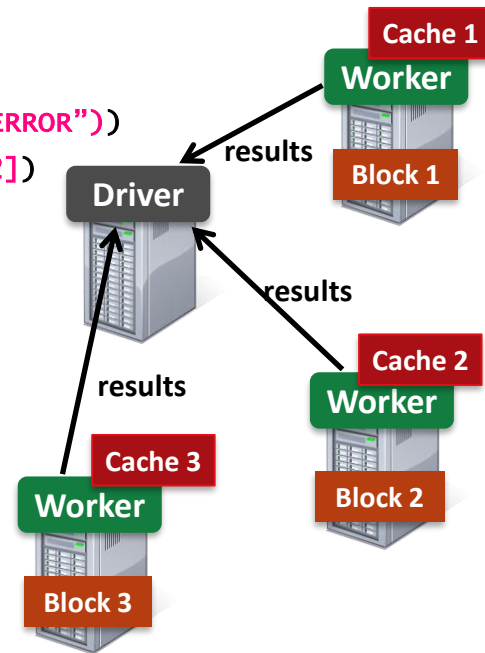


Caching

Log Mining example: Load error messages from a log into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda s: s.startswith("ERROR"))
messages = errors.map(lambda s: s.split("\t")[2])
messages.cache()
```

```
messages.filter(lambda s: "mysql" in s).count()
messages.filter(lambda s: "php" in s).count()
```



Caching

Log Mining example: Load error messages from a log into memory, then interactively search for various patterns

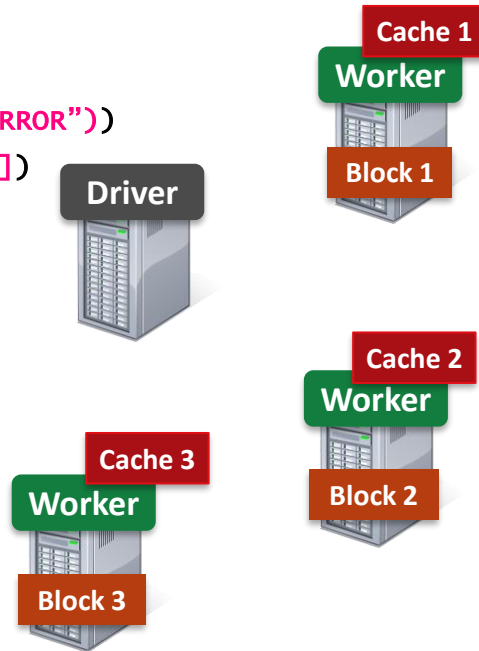
```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda s: s.startswith("ERROR"))
messages = errors.map(lambda s: s.split("\t")[2])
messages.cache()
```

```
messages.filter(lambda s: "mysql" in s).count()
messages.filter(lambda s: "php" in s).count()
```

Cache your data → Faster Results

Full-text search of Wikipedia

- 60GB on 20 EC2 machines
- 0.5 sec from mem vs. 20s for on-disk





Caching

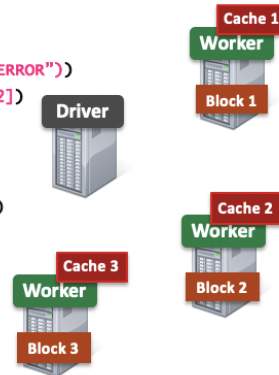
- `cache()` : saves an RDD to memory (of each worker node).
- `persist(options)` : can be used to save an RDD to memory, disk, or off-heap memory
- When should we cache or not cache an RDD?
 - When it is expensive to compute and needs to be re-used multiple times.
 - If worker nodes have not enough memory, they will evict the “least recently used” RDDs. So, be aware of memory limitations when caching.

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda s: s.startswith("ERROR"))
messages = errors.map(lambda s: s.split("\t")[2])
messages.cache()
```

```
messages.filter(lambda s: "mysql" in s).count()
messages.filter(lambda s: "php" in s).count()
```

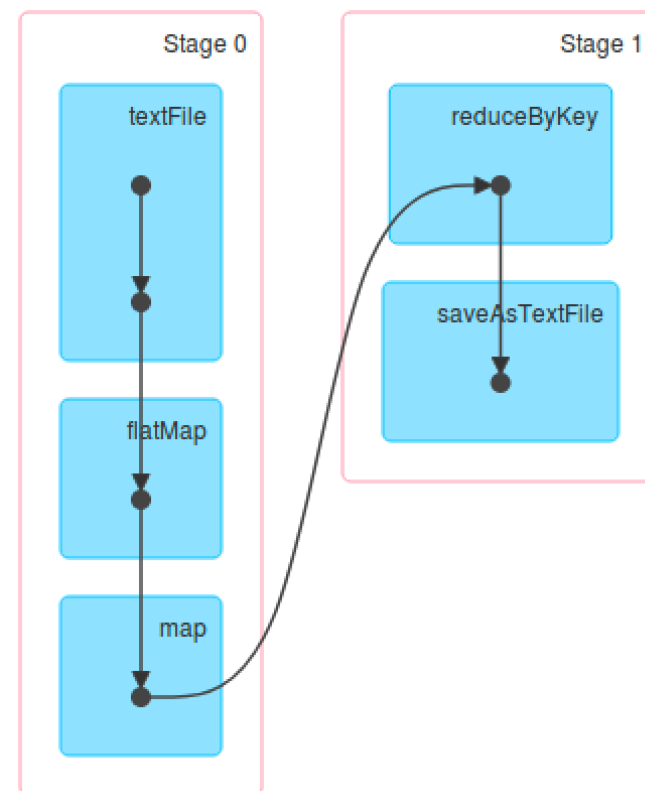
Cache your data → Faster Results
Full-text search of Wikipedia

- 60GB on 20 EC2 machines
- 0.5 sec from mem vs. 20s for on-disk



Directed Acyclic Graph (DAG)

- Internally, Spark creates a graph (“directed acyclic graph”) which represents all the RDD objects and how they will be transformed.
- Transformations construct this graph; actions trigger computations on it.

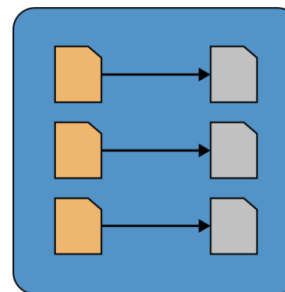


```
val file = sc.textFile("hdfs://...")  
  
val counts = file.flatMap(line => line.split(" "))  
                  .map(word => (word, 1))  
                  .reduceByKey(_ + _)  
counts.save("...")
```

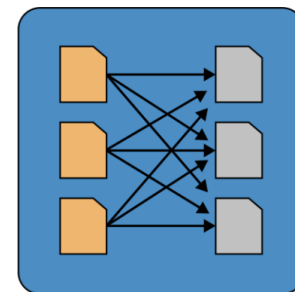
Narrow and Wide Dependencies

- **Narrow dependencies** are where each partition of the parent RDD is used by at most 1 partition of the child RDD
 - E.g. map, flatMap, filter, contains
- **Wide dependencies** are the opposite (each partition of parent RDD is used by multiple partitions of the child RDD)
 - E.g. reduceByKey, groupBy, orderBy
- In the DAG, consecutive narrow dependencies are grouped together as “**stages**”.
- **Within stages**, Spark performs consecutive transformations on the same machines.
- **Across stages**, data needs to be **shuffled**, i.e. exchanged across partitions, in a process very similar to map-reduce, which involves writing intermediate results to disk
- Minimizing shuffling is good practice for improving performance.

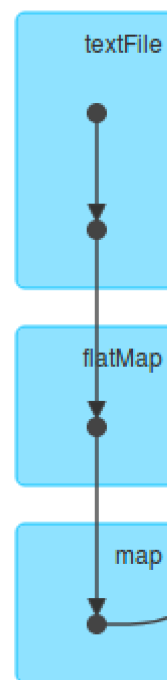
Narrow Dependencies



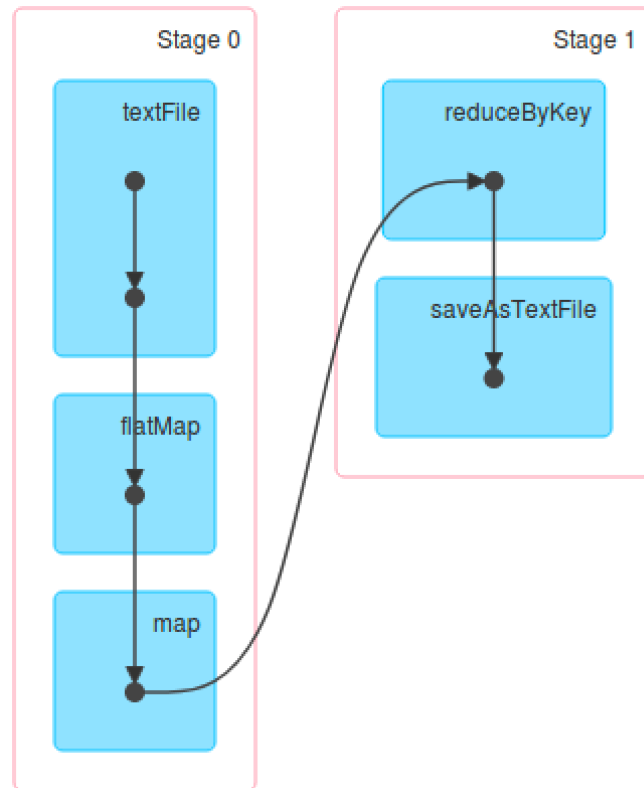
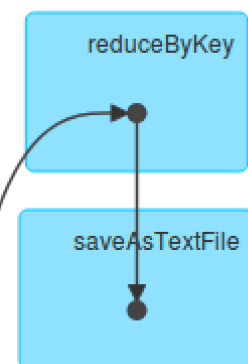
Wide Dependencies



Stage 0



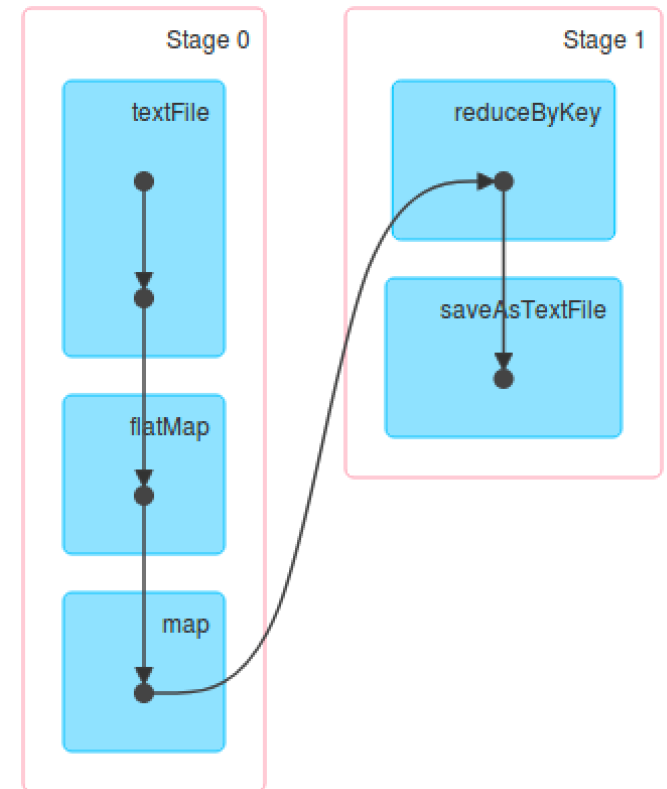
Stage 1





Lineage and Fault Tolerance

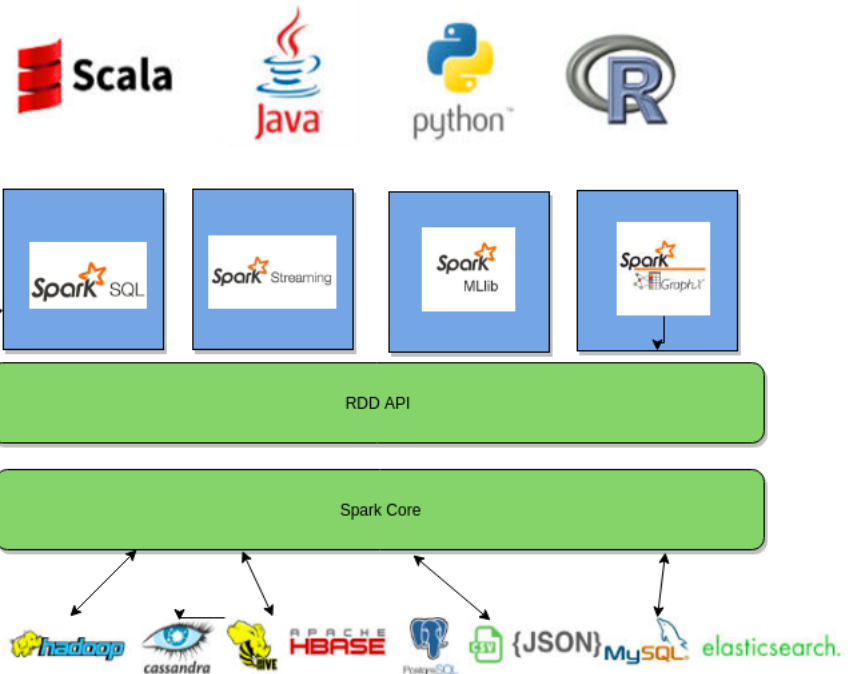
- Unlike Hadoop, Spark does not use replication to allow fault tolerance. Why?
 - Spark tries to store all the data in memory, not disk. Memory capacity is much more limited than disk, so simply duplicating all data is expensive.
- **Lineage approach:** if a worker node goes down, we replace it by a new worker node, and use the graph (DAG) to recompute the data in the lost partition.
 - Note that we only need to recompute the RDDs from the lost partition.



Today's Plan

- Introduction and Basics
- Working with RDDs
- Caching and DAGs

- **DataFrames and Datasets**



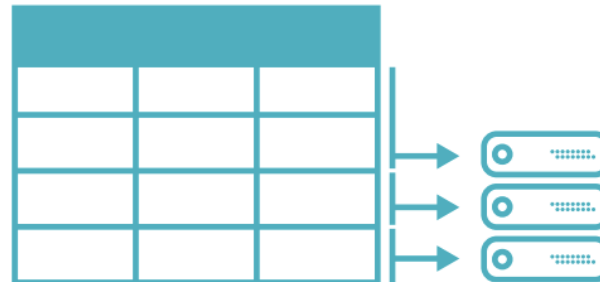
DataFrames

- A DataFrame represents a table of data, similar to tables in SQL, or DataFrames in pandas.
- Compared to RDDs, this is a higher level interface, e.g. it has transformations that resemble SQL operations.
 - DataFrames are the recommended interface for working with Spark – they are easier to use than RDDs and almost all tasks can be done with them, while only rarely using the RDD functions.
 - However, all DataFrame operations are still ultimately compiled down to RDD operations by Spark.

Spreadsheet on a
single machine



Table or DataFrame partitioned
across servers in data center



DataFrames: example

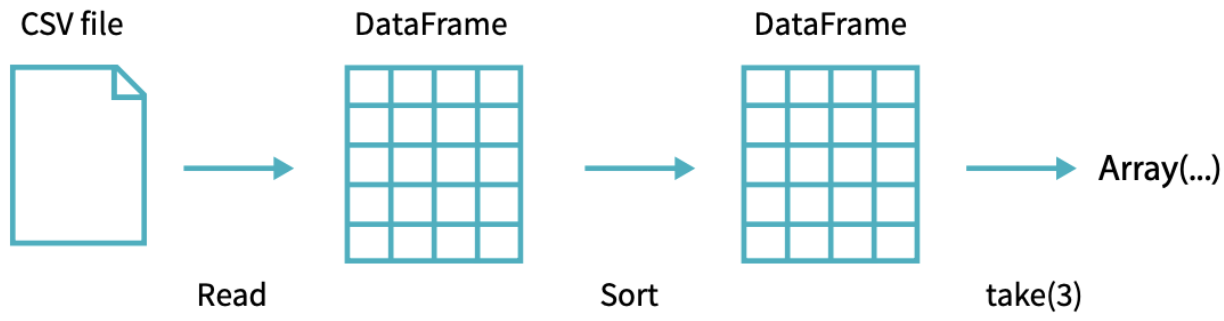
```
flightData2015 = spark\  
.read\  
.option("inferSchema", "true")\  
.option("header", "true")\  
.csv("/mnt/defg/flight-data/csv/2015-summary.csv")
```

- Reads in a DataFrame from a CSV file.

```
flightData2015.sort("count").take(3)
```

- Sorts by 'count' and output the first 3 rows (action)

```
Array([United States,Romania,15], [United States,Croatia...
```



DataFrames: transformations

- An easy way to transform DataFrames is to use SQL queries. This takes in a DataFrame and returns a DataFrame (the output of the query).

```
flightData2015.createOrReplaceTempView("flight_data_2015")
maxSql = spark.sql("""
SELECT DEST_COUNTRY_NAME, sum(count) as destination_total
FROM flight_data_2015
GROUP BY DEST_COUNTRY_NAME
ORDER BY sum(count) DESC
LIMIT 5
""")
maxSql.collect()
```

DataFrames: DataFrame interface

- We can also run the exact same query as follows:

```
from pyspark.sql.functions import desc
flightData2015\
.groupBy("DEST_COUNTRY_NAME")\
.sum("count")\
.withColumnRenamed("sum(count)", "destination_total")\
.sort(desc("destination_total"))\
.limit(5)\
.collect()
```

- Generally, these transformation functions (groupBy, sort, ...) take in either strings or “column objects”, which represent columns.
 - For example, “desc” here returns a column object.

Datasets

- Datasets are similar to DataFrames, but are type-safe.
 - In fact, in Spark (Scala), DataFrame is just an alias for Dataset[Row]
 - However, Datasets are not available in Python and R, since these are dynamically typed languages

```
case class Flight(DEST_COUNTRY_NAME: String, ORIGIN_COUNTRY_NAME: String,
count: BigInt)

val flightsDF = spark.read.parquet("/mnt/defg/flight-data/parquet/2010-
summary.parquet/")

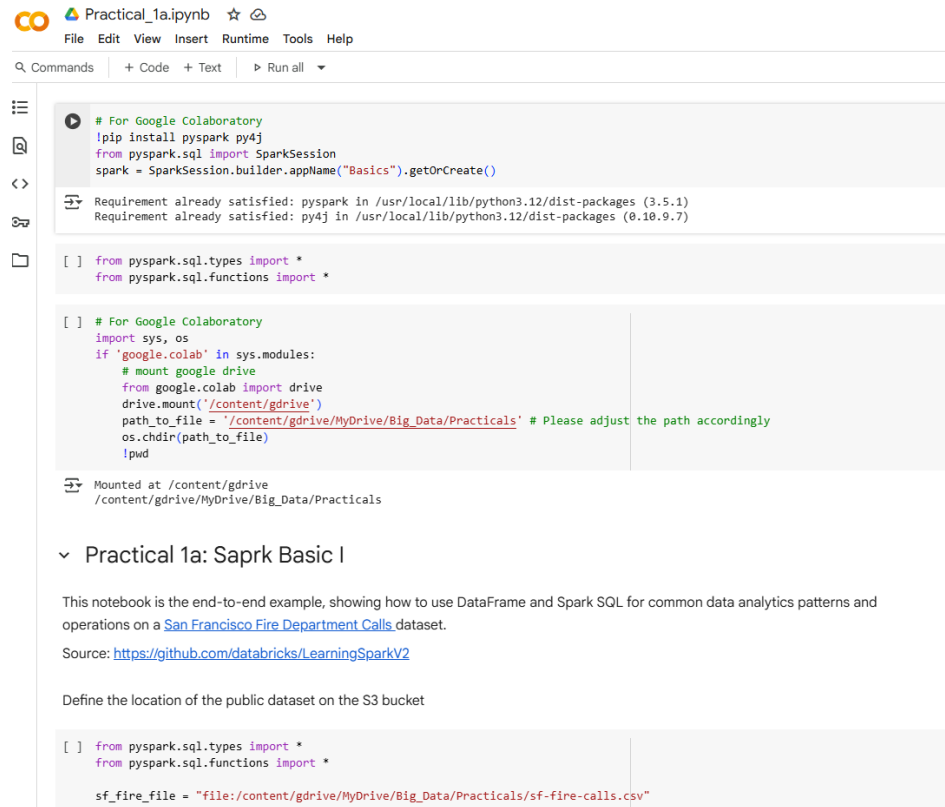
val flights = flightsDF.as[Flight]

flights.collect()
```

- The Dataset `flights` is type safe – its type is the “Flight” class.
- Now when calling `collect()`, it will also return objects of the “Flight” class, instead of Row objects.

Example: Spark Notebooks in Google Colab

- You can download all the Practical notebooks from Canvas/Files/Practicals.zip
- Support most Spark functions including Spark SQL, Spark ML, Spark Streaming, Graphframes and etc.



```
Practical_1a.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all ▼

# For Google Colaboratory
!pip install pyspark py4j
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Basics").getOrCreate()

Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (3.5.1)
Requirement already satisfied: py4j in /usr/local/lib/python3.12/dist-packages (0.10.9.7)

[ ] from pyspark.sql.types import *
    from pyspark.sql.functions import *

[ ] # For Google Colaboratory
import sys, os
if 'google.colab' in sys.modules:
    # mount google drive
    from google.colab import drive
    drive.mount('/content/gdrive')
    path_to_file = '/content/gdrive/MyDrive/Big_Data/Practicals' # Please adjust the path accordingly
    os.chdir(path_to_file)
    !pwd

Mounted at /content/gdrive
/content/gdrive/MyDrive/Big_Data/Practicals

▼ Practical 1a: Saprk Basic I

This notebook is the end-to-end example, showing how to use DataFrame and Spark SQL for common data analytics patterns and operations on a San Francisco Fire Department Calls dataset.

Source: https://github.com/databricks/LearningSparkV2

Define the location of the public dataset on the S3 bucket

[ ] from pyspark.sql.types import *
    from pyspark.sql.functions import *

sf_fire_file = "file:/content/gdrive/MyDrive/Big_Data/Practicals/sf-fire-calls.csv"
```

Example: Spark Notebooks in Databricks

- You need to sign up a Databricks free edition account (**with limited functions**) <https://www.databricks.com/learn/free-edition>

Practical 1a: Spark Basic I

This notebook is the end-to-end example, showing how to use DataFrame and Spark SQL for common

Source: <https://github.com/databricks/LearningSparkV2>

Cmd 2

Inspect location where the SF Fire Department Fire calls data set is stored in the public dataset S3 bucket

Cmd 3

```
1 %fs ls /databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv
```

Table +

	path	name	size
1	dbfs:/databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv	sf-fire-calls.csv	1137925359

1 row | 2.04 seconds runtime

Command took 2.04 seconds -- by aixin@comp.nus.edu.sg at 2/18/2023, 2:32:32 PM on Test

Cmd 4

Define the location of the public dataset on the S3 bucket

Cmd 5

```
1 from pyspark.sql.types import *
2 from pyspark.sql.functions import *
3
4 sf_fire_file = "/databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv"
```

Command took 0.00 seconds -- by aixin@comp.nus.edu.sg at 2/18/2023, 2:32:45 PM on Test

Practical 1b: Spark Basic II

The power of Spark SQL is that it contains many DataFrame Operations (also known as Untyped Dataset operations) for DataFrames and Spark SQL.

Source: <https://github.com/databricks/LearningSparkV2>

Cmd 2

```
1 from pyspark.sql.functions import expr
2
3 # Set File Paths
4 delays_path = "/databricks-datasets/learning-spark-v2/flights/departuredelays.csv"
5 airports_path = "/databricks-datasets/learning-spark-v2/flights/airport-codes-na.txt"
6
7 # Obtain airports dataset
8 airports = spark.read.options(header="true", inferSchema="true", sep="t").csv(airports_path)
9 airports.createOrReplaceTempView("airports_na")
10
11 # Obtain departure Delays data
12 delays = spark.read.options(header="true").csv(delays_path)
13 delays = (delays
14          .withColumn("delay", expr("CAST(delay as INT) as delay"))
15          .withColumn("distance", expr("CAST(distance as INT) as distance")))
16
17 delays.createOrReplaceTempView("departureDelays")
18
19 # Create temporary small table
20 foo = delays.filter(expr("""
21     origin == 'SEA' AND
22     destination == 'SFO' AND
23     date like '01010%' AND
24     delay > 0"""))
25
26 foo.createOrReplaceTempView("foo")
```

Demo_1: Spark Web UI

```
1 df1 = spark.range(2, 100000000, 2)
2 df2 = spark.range(2, 100000000, 4)
3 df3 = df1.join(df2, ["id"])
4 df3.count()
```

▼ (4) Spark Jobs

▼ Job 0 [View](#) (Stages: 1/1)

Stage 1: 8/8 ⓘ

▼ Job 1 [View](#) (Stages: 1/1)

Stage 0: 8/8 ⓘ

▼ Job 2 [View](#) (Stages: 1/1, 2 skipped)

Stage 2: 0/8 ⓘ skipped

Stage 3: 0/8 ⓘ skipped

Stage 4: 8/8 ⓘ

▼ Job 3 [View](#) (Stages: 1/1, 3 skipped)

Stage 5: 0/8 ⓘ skipped

Stage 6: 0/8 ⓘ skipped

Stage 7: 0/8 ⓘ skipped

Stage 8: 1/1 ⓘ

▶  df1: pyspark.sql.dataframe.DataFrame = [id: long]

▶  df2: pyspark.sql.dataframe.DataFrame = [id: long]

▶  df3: pyspark.sql.dataframe.DataFrame = [id: long]

Out[1]: 25000000

```
1 df1.show(10)
```

▶ (1) Spark Jobs

+---+

| id|

+---+

| 2|

| 4|

| 6|

| 8|

| 10|

| 12|

| 14|

| 16|

| 18|

| 20|

+---+

```
1 df2.show(10)
```

▶ (1) Spark Jobs

+---+

| id|

+---+

| 2|

| 6|

| 10|

| 14|

| 18|

| 22|

| 26|

| 30|

| 34|

| 38|

+---+

```
1 df3.show(10)
```

▶ (3) Spark Jobs

+---+

| id|

+---+

| 22|

| 26|

| 34|

| 50|

| 54|

| 94|

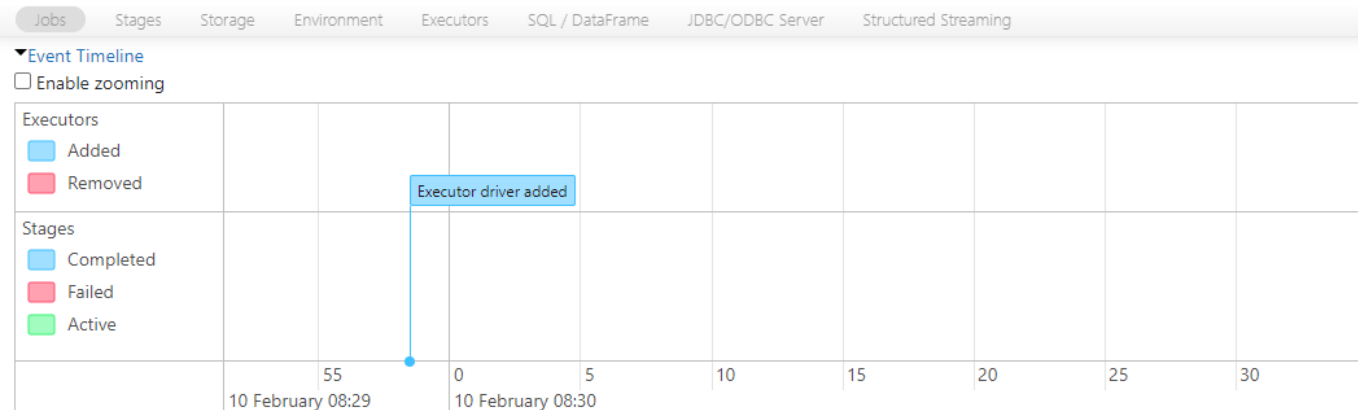
| 110|

| 126|

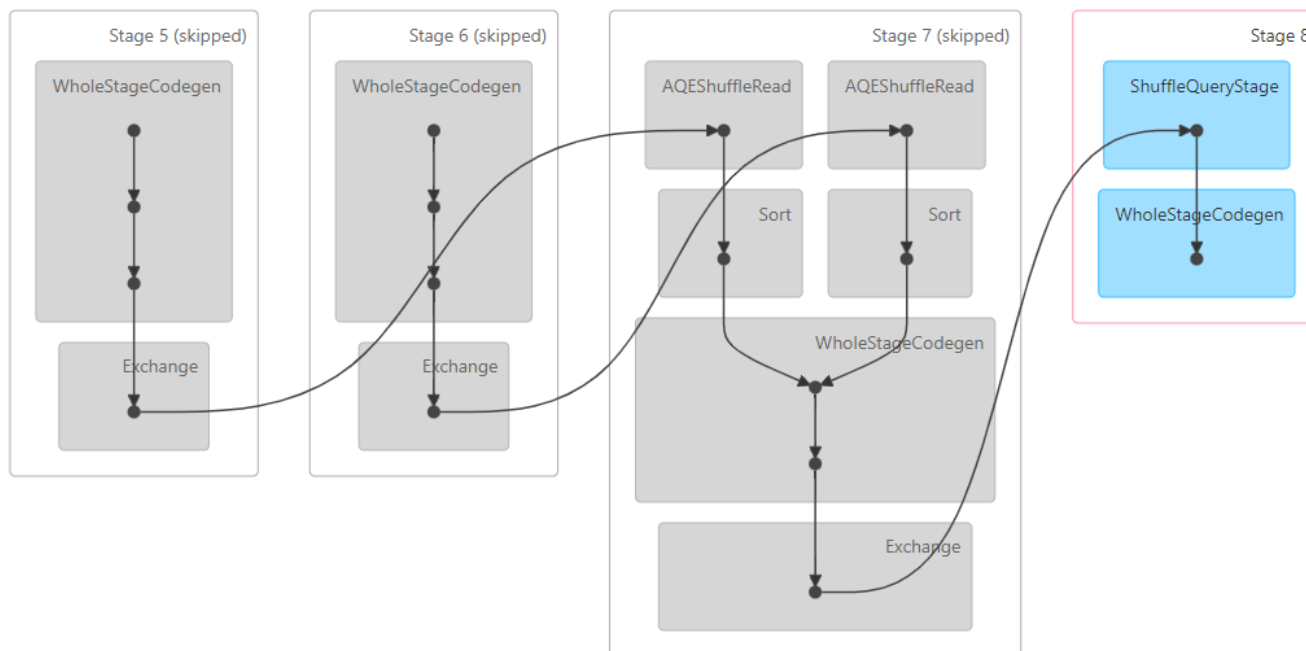
| 130|

| 190|

+---+



▼ DAG Visualization



Stages for All Jobs

Completed Stages: 4

Skipped Stages: 5

▼ Fair Scheduler Pools (1)

Pool Name	Minimum Share	Pool Weight	Active Stages	Running Tasks	SchedulingMode
default	0	1	0	0	FIFO

▼ Completed Stages (4)

Page: 11 Pages, Jump to 1. Show 100 items in a page. G

Stage Id ▾	Pool Name	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
8	3168022962293687376	df1 = spark.range(2, 10000000, 2) df2 = spark.r... count at NativeMethodAccessorImpl.java:0	2023/02/10 08:31:15	0.3 s	1/1			472.0 B	
4	3168022962293687376	df1 = spark.range(2, 10000000, 2) df2 = spark.r... \$anonfun\$withThreadLocalCaptured\$1 at CompletableFuture.java:1604	2023/02/10 08:31:05	9 s	8/8			36.5 MiB	472.0 B
1	3168022962293687376	df1 = spark.range(2, 10000000, 2) df2 = spark.r... \$anonfun\$withThreadLocalCaptured\$1 at CompletableFuture.java:1604	2023/02/10 08:30:53	3 s	8/8				12.2 MiB
0	3168022962293687376	df1 = spark.range(2, 10000000, 2) df2 = spark.r... \$anonfun\$withThreadLocalCaptured\$1 at CompletableFuture.java:1604	2023/02/10 08:30:52	8 s	8/8				24.3 MiB

Executors

► Show Additional Metrics

Summary

	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Excluded
Active(1)	0	0.0 B / 3.9 GiB	0.0 B	8	0	0	25	25	5.6 min (4 s)	0.0 B	36.5 MiB	36.5 MiB	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0
Total(1)	0	0.0 B / 3.9 GiB	0.0 B	8	0	0	25	25	5.6 min (4 s)	0.0 B	36.5 MiB	36.5 MiB	0

Executors

Show 20 entries

Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Thread Dump	Heap Histogram	Exec Loss Reason
driver	10.172.213.39:43725	Active	0	0.0 B / 3.9 GiB	0.0 B	8	0	0	25	25	5.6 min (4 s)	0.0 B	36.5 MiB	36.5 MiB	Thread Dump	Heap Histogram	

SQL / DataFrame

Completed Queries: 5

▼ Completed Queries (5)

Page: 1 1 Pages. Jump to 1 . Show 100 item

ID	Description	Submitted	Duration	Job IDs	Sub Execution IDs
4	show tables in 'default'	2023/02/10 08:31:23	31 ms		
3	show tables in 'default'	2023/02/10 08:31:22	0.1 s		
2	show databases	2023/02/10 08:31:21	52 ms		
1	df1 = spark.range(2, 10000000, 2) df2 = spark.r...	2023/02/10 08:30:48	27 s	[0][1][2][3]	
0	show databases	2023/02/10 08:30:39	41 s		

Storage

Parquet IO Cache

Data Read from External Filesystem (All Formats)	Data Read from IO Cache (Cache Hits, Compressed)	Data Written to IO Cache (Compressed)	Cache Misses (Compressed)	True Cache Misses	Partial Cache Misses	Rescheduling Cache Misses	Cache Hit Ratio	Number of Local Scan Tasks	Number of Rescheduled Scan Tasks	Cache Metadata Manager Peak Disk Usage
0.0 B	0.0 B	0.0 B	0.0 B	0.0 B	0.0 B	0.0 B	0 %	0	0	0.0 B

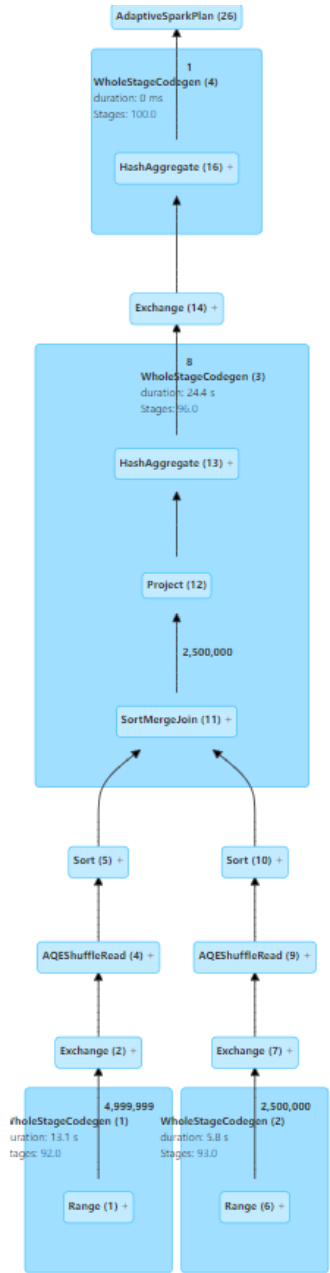
Environment

▼ Runtime Information

Name	Value
Java Home	/usr/lib/jvm/zulu8-ca-amd64/jre
Java Version	1.8.0_345 (Azul Systems, Inc.)
Scala Version	version 2.12.14

▼ Spark Properties

Name	Value
eventLog.rollOverIntervalSeconds	900
libraryDownload.sleepIntervalSeconds	5
libraryDownload.timeoutSeconds	180
spark.akka.frameSize	256
spark.app.id	local-1676017796375
spark.app.name	Databricks Shell
spark.app.startTime	1676017791391
spark.cleaner.referenceTracking.blocking	false
spark.databricks.acl.client	com.databricks.spark.sql.acl.client.SparkSqlAclClient
spark.databricks.acl.provider	com.databricks.sql.acl.ReflectionBackedAclProvider
spark.databricks.acl.scim.client	com.databricks.spark.sql.acl.client.DriverToWebappScimClient
spark.databricks.automi.serviceEnabled	true
spark.databricks.cloudProvider	AWS
spark.databricks.cloudfetch.hasRegionSupport	true
spark.databricks.cloudfetch.requesterClassName	***** (redacted)
spark.databricks.clusterSource	UI



Demo_2: Caching Data

```
1 from pyspark.sql.functions import col
2
3 df = spark.range(1 * 10000000).toDF("id").withColumn("square", col("id") * col("id"))
4 df.cache().count()
```

► (3) Spark Jobs

►  df: pyspark.sql.dataframe.DataFrame = [id: long, square: long]

Out[1]: 10000000

Command took 10.90 seconds -- by aixin@comp.nus.edu.sg at 8/24/2023, 3:40:21 PM on MyCluster

Jobs	Stages	Storage	Environment	Executors	SQL / DataFrame	JDBC/ODBC Server	Structured Streaming	
------	--------	---------	-------------	-----------	-----------------	------------------	----------------------	---

Storage

Parquet IO Cache

Data Read from External Filesystem (All Formats)	Data Read from IO Cache (Cache Hits, Compressed)	Data Written to IO Cache (Compressed)	Cache Misses (Compressed)	True Cache Misses	Rescheduling Cache Misses	Cache Hit Ratio	Number of Local Scan Tasks	Number of Rescheduled Scan Tasks	Cache Metadata Manager Peak Disk Usage
172.3 MiB	0.0 B	0.0 B	0.0 B	0.0 B	0.0 B	0 %	0	0	0.0 B

▼ RDDs

ID	RDD Name	Storage Level	Cached Partitions	Fraction Cached	Size in Memory	Size on Disk
190	*(1) Project [id#814L, (id#814L * id#814L) AS square#818L] +- *(1) Range (0, 10000000, step=1, splits=8)	Disk Memory Deserialized 1x Replicated	8	100%	86.2 MiB	0.0 B

```
1 df.count()
```

► (2) Spark Jobs

Out[2]: 10000000

Command took 1.26 seconds -- by aixin@comp.nus.edu.sg at 8/24/2023, 3:40:46 PM on MyCluster

Demo_1 in Databricks free edition

No Spark UI and only has Query Profile

The screenshot displays the Databricks free edition interface. On the left is a sidebar with navigation options: New, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie, Alerts, Query History, SQL Warehouses, Data Engineering, Job Runs, Data Ingestion, AI/ML, Playground, Experiments, Features, Models, and Serving. The main workspace shows a notebook titled 'Demo_1' with a Python tab. The notebook code defines three DataFrames (df1, df2, df3) and counts them. The execution history shows a successful run of 'df3.count()' with a duration of 2 s 864 ms. Below the code, a table lists the statements and their durations. The right sidebar shows the 'Finished' status of the query, including the ID 'e1ec65fb-b27e-4b00-a21d-95823b50446c', the query text 'df3.count()', and a 'See query profile' link. The query profile section shows the total wall-clock duration of 2 s 864 ms, with a breakdown of 607 ms for 'Optimizing query & pruning files' and 2 s 257 ms for 'Executing'. The query source is identified as 'Demo_1' / 'Cell(ID: 725...21158133): 4'.

Demo_1:
Source: <https://github.com/databricks/Spark-The-Definitive-Guide>

```
df1 = spark.range(2, 10000000, 2)
df2 = spark.range(2, 10000000, 4)
df3 = df1.join(df2, ["id"])
df3.count()
```

Statement	Started At	Tasks	Duration
L4 > df3.count()	Sep 03, 2025, 10:35 AM	25/25 completed	2 s 864 ms

df1: pyspark.sql.connect.dataframe.DataFrame = [id: long]
df2: pyspark.sql.connect.dataframe.DataFrame = [id: long]
df3: pyspark.sql.connect.dataframe.DataFrame = [id: long]
2500000

```
df1.show(10)
```

++
| id |
++
| 2 |
| 4 |

Finished
Xin Ai · Serverless compute
ID: e1ec65fb-b27e-4b00-a21d-95823b50446c
df3.count()
See query text

Bytes read: 0, Bytes written: 0
Rows read: 0, Rows written: 0
Files read: 0, Files written: 0
See query profile >

See longest operations for this query

Query wall-clock duration
Total wall-clock duration: 2 s 864 ms
Optimizing query & pruning files: 21% (607 ms)
Executing: 79% (2 s 257 ms)

Start time: Sep 03, 2025, 10:35:43 AM GMT+08:00
End time: Sep 03, 2025, 10:35:46 AM GMT+08:00
Result fetching by client: 283 ms

Query Source
> Demo_1 / Cell(ID: 725...21158133): 4

Aggregated task time
Task total time: 7 s 1 s
See query profile

ID: e1ec65fb-b27e-4b00-a21d-95823b50446c

df3.count()
See query text

Bytes read
0

Bytes written
0

Rows read
0

Rows written
0

Files read
0

Files written
0

See query profile >

See longest operations for this query

Query wall-clock duration ⓘ

Total wall-clock duration2 s 864 ms

Optimizing query & pruning files ⓘ21%607 ms

Executing ⓘ79%2 s 257 ms

Start timeSep 03, 2025, 10:35:43 AM GMT+08:00

End timeSep 03, 2025, 10:35:46 AM GMT+08:00

Result fetching by client ⓘ283 ms

Query Source

> Demo_1 / Cell(ID: 725...21158133): 4

Aggregated task time ⓘ

Tasks total time7.51 s

Tasks time in Photon42 %

IO

Rows returned1

Rows read0

Bytes read0 bytes

Bytes pruned0 bytes

See query profile

```

graph BT
    S10["#10 Range  
2.50 M rows  
1 ms"] -- "2.50 M rows" --> S9["#9 Shuffle  
237 ms"]
    S9 -- "2.50 M rows" --> S6["#6 Inner Join  
on id, id  
2.20 s"]
    S8["#8 Range  
10 ms"] -- "5.00 M rows" --> S7["#7 Shuffle  
661 ms"]
    S7 -- "5.00 M rows" --> S6
    S6 -- "2.50 M rows" --> S5["#5 Aggregate  
1 ms"]
    S5 -- "8 rows" --> S4["#4 Shuffle  
3 ms"]
    S4 -- "8 rows" --> S3["#3 Aggregate  
0 ms"]
    S3 -- "1 rows" --> S2["#2 Columnar To Row  
24 ms"]
    S2 -- "1 rows" --> S1["#1 Result Query Stage  
0 ms"]
  
```

52

NUS Confidential

Take-away in the AI Era

- (Why Spark system design decisions still require humans)

- **What AI cannot decide for you (yet)**

- Whether Spark system is suitable for your organization?
- Which RDDs should be cached? Cached to memory/hard disk?
- Translate a business request to an architecture change?

E.g. let the business users get the results faster → repartition the data to minimize the wide dependencies transformation

- **What AI can assist you**

- Generate Spark codes based on your instructions
- Help you debug the codes and figure out the errors

Acknowledgements

- CS4225 slides by He Bingsheng and Bryan Hooi
- Jules S. Damji, Brooke Wenig, Tathagata Das & Denny Lee, “Learning Spark: Lightning-Fast Data Analytics”
- Databricks, “The Data Engineer’s Guide to Spark”
- <https://www.pinterest.com/pin/739364463807740043/>
- <https://untitled-life.github.io/blog/2018/12/27/wide-vs-narrow-dependencies/>